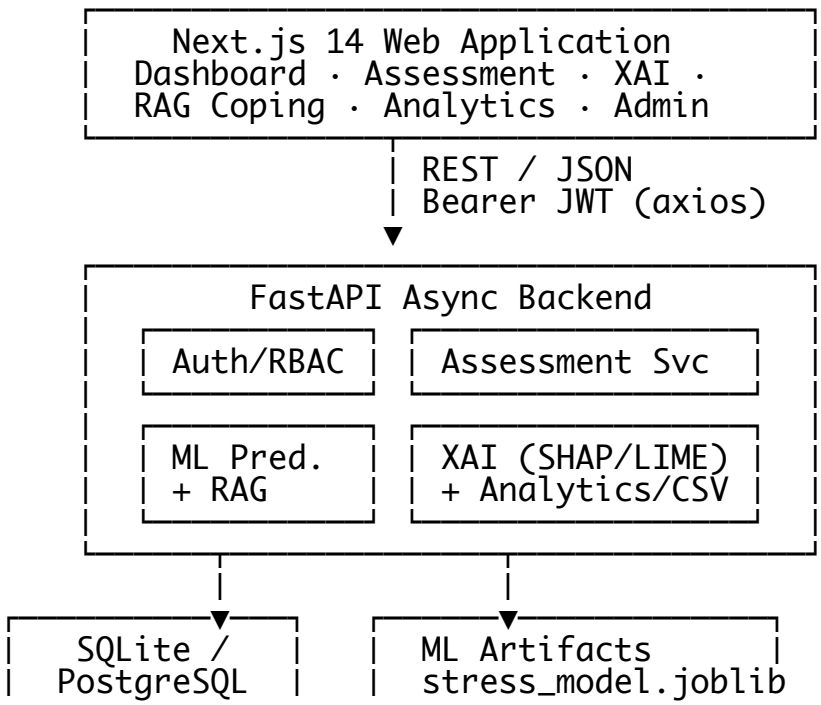
Psychological Stress AI
System Architecture, Workflow & Data Guide

Psychological Stress AI — System Architecture, Workflow & Data Guide
> Complete technical reference: how the application works, end-to-end workflow,
> every dataset it uses, the database server, and the full architecture.
---
1. Executive Summary
The Psychological Stress AI platform is a full-stack, multi-modal, explainable AI application for psychological stress assessment. A user completes a PSS-10 questionnaire plus physiological, sleep, and workload inputs. The system scores the answers, runs a trained gradient-boosted machine learning model to predict a stress level (Low / Moderate / High / Severe), explains the prediction with SHAP and LIME, and retrieves evidence-based coping interventions with a Retrieval-Augmented Generation (RAG) engine.

Component	Technology	Role
Frontend	Next.js 14 (React, TailwindCSS, Recharts, Framer Motion)	Web UI on port 3000
Backend	FastAPI (async, Python 3.9+)	REST API on port 8000
ML Engine	XGBoost / RandomForest / GradientBoosting + SHAP + LIME	Prediction & explainability
RAG Engine	TF-IDF + Cosine Similarity	Coping intervention retrieval
Database	SQLite (aiosqlite) for local dev; PostgreSQL supported	Relational store
Auth	JWT (HS256) + bcrypt (12 rounds)	Security & RBAC

---
2. High-Level Architecture



```
                    |  6 tables    |   |  preprocessor.joblib
                    └──────────────┘   |  rag_engine.joblib  |
                                       |  eval_metrics.json  |
                                       └─────────────────────┘
```

2.1 Frontend (Next.js 14 App Router)
src/app/ – pages: login, register, dashboard, assessment, predictions, explainability, rag-coping, analytics, admin, settings.
src/components/ – Navbar, Sidebar, GlassCard (glassmorphic UI), ShapBarChart (Recharts), LimeImpactList, RagInterventionCard, StressGauge, StatCard.
src/lib/api.ts – central axios client with JWT interceptor (reads token from localStorage, clears on 401).
src/lib/auth.tsx – auth context protecting pages by role.
2.2 Backend (FastAPI)
backend/app/main.py – app factory, CORS, lifespan DB init, /health, global exception handler.
backend/app/api/v1/api.py – router registration:
/auth – login, register, me (JWT + bcrypt, RBAC)
/users – user management
/assessments – create / list / get stress assessments
/predictions – create prediction, list user predictions
/explainability/{prediction_id} – SHAP + LIME explanations
/rag – RAG coping interventions
/analytics – user + admin analytics
/admin – user management, system analytics (admin only)
/reports/csv – CSV report export
backend/app/core/ – config.py (pydantic settings, loads .env), security.py (JWT, bcrypt), database.py (async SQLAlchemy engine/session).
backend/app/services/ – ml_service.py (orchestrates prediction + XAI logging), stress_service.py, rag_service.py, analytics_service.py, report_service.py, auth_service.py, user_service.py.
---
3. End-to-End Workflow
3.1 User Journey (happy path)

```
1. Register/Login  ──► JWT token stored in localStorage
2. Dashboard       ──► GET /analytics/user (trends, gauges)
3. Assessment      ──► 4-step wizard:
                        Step 1: PSS-10 questionnaire (10 items, 0-4 scale)
                        Step 2: heart_rate, hrv_sdnn, sleep_hours,
                                sleep_efficiency, physical_activity_min
                        Step 3: work_hours, screen_time_hours, breaks_per_day,
                                sentiment_score, anxiety_score
                        Step 4: review & submit
4. Prediction      ──► POST /assessments/  (server computes total_pss,
                        reverse-scoring items 4,5,7,8)
                   ──► POST /predictions/  (ML inference, 96-97% accuracy)
                   ──► redirect /predictions?id=<id>
5. Explainability ──► GET /explainability/<pred_id> (SHAP + LIME)
6. Coping Plan    ──► POST /rag/ (TF-IDF retrieval, top-3 interventions)
7. Analytics      ──► GET /analytics/user (charts)
8. Export         ──► GET /reports/csv (download)
```

3.2 Prediction & XAI Pipeline (server side)

```
assessment record
      |
      ▼
raw_input dict (21 fields: pss_q1..q10, total_pss, HR, HRV, sleep, workload…)
      |
      ▼
engineer_stress_features()        → 4 engineered features:
```

```
                                    hrv_to_hr_ratio
                                    sleep_deficiency_index
                                    work_stress_factor
                                    composite_strain_index
      |
      ▼
StressDataPreprocessor            → impute (median) + RobustScaler
      |                             (25-dimensional feature vector)
      ▼
Trained classifier (GradientBoostingClassifier, best of 3 via CV)
      |
      ├──► stress_level (Low/Moderate/High/Severe)
      ├──► confidence_score + class_probabilities
      ├──► SHAP: per-feature attribution, top-7 drivers
      ├──► LIME: local surrogate if-then rules
      └──► RAG: interventions matched on SHAP drivers + stress level
      |
      ▼
Persist: predictions row + explainability_logs row (JSON)
```

3.3 RAG Coping Workflow

Build query: "Stress level <level> drivers <top SHAP driver names> <user text>"
TfidfVectorizer transform (fit on 6 intervention documents).
Cosine similarity query → document.
Domain boost: +0.25 per SHAP driver that matches an intervention's target_drivers.
Return top-3 interventions with relevance_score; logged to rag_coping_logs.

3.4 Security Workflow

Passwords hashed with bcrypt (12 rounds) — never stored in plain text.
Login returns JWT (HS256, 7-day expiry, ACCESS_TOKEN_EXPIRE_MINUTES=10080).
Every protected route validates the bearer token via get_current_user.
RBAC roles: user (own data), clinician (patient analytics), admin (user management, system analytics).
Auth events, predictions, and RAG queries are written to audit_logs.

---

4. Datasets Used by the Application

4.1 Synthetic Training Dataset (ML model)

Item	Detail
Generator	ml_engine/data/generate_synthetic_stress_dataset.py
Artifact	ml_engine/data/synthetic_stress_data.csv (3,000 records)
Features	10 PSS-10 items (0–4), total_pss (0–40, reverse-scored), heart rate (50–120 bpm), HRV SDNN (15–120 ms), sleep hours, sleep efficiency, physical activity, work hours, screen time, breaks, sentiment (–1 to +1), anxiety (1–10)
Target	stress_level 0–3 derived from a weighted composite index (PSS 40%, HRV 25%, sleep 20%, anxiety 15%)
Use	Trains/cross-validates XGBoost, RandomForest, GradientBoosting; 80/20 stratified split

Class distribution thresholds: >= 0.35 Moderate, >= 0.58 High, >= 0.78 Severe.
Model quality (test set): accuracy 97%, macro F1 89.3%, ROC-AUC 99.5% (GradientBoosting — currently selected best model).

4.2 RAG Knowledge Base (coping interventions)

Item	Detail

File
ml_engine/rag/rag_knowledge_base.json
Records
6 evidence-based interventions
Categories
Autonomic Nervous System Regulation, CBT, Mindfulness & Affective Regulation, Physical Activity & Somatic Discharge, Sleep Hygiene & Recovery, Workload & Screen Ergonomics
Fields
id, category, title, summary, protocol (steps), evidence_base, difficulty, duration_min, target_drivers (mapped to SHAP feature names)
Use
Indexed with TF-IDF at training time, serialized into rag_engine.joblib

4.3 Application / Seed Data (SQLite)
database/seed_data.py – seeds 3 demo accounts:
user@stressai.com / User@2026 (user)
admin@stressai.com / Admin@StressAI2026 (admin)
dr.sarah@clinic.com / Clinician@2026 (clinician)
database/init.sql – full PostgreSQL DDL (same schema as SQLite, plus
  CHECK constraints, JSONB, indexes, seed admin).
Runtime data generated by users: assessments, predictions, XAI logs, RAG
  logs, audit logs.
4.4 ML Model Artifacts (`ml_engine/models/`)
File
Contents
stress_model.joblib
Trained best classifier + name + CV results
preprocessor.joblib
Fitted median imputer + RobustScaler
rag_engine.joblib
Knowledge base + fitted TF-IDF vectorizer
eval_metrics.json
Accuracy, F1, precision, recall, ROC-AUC

---
5. Database Server & Schema
5.1 Current setup
Local development: SQLite file stress_ai.db (.env:
  DATABASE_URL=sqlite+aiosqlite:///./stress_ai.db), ~116 KB, auto-created at
  backend startup via Base.metadata.create_all.
Production/containerized: PostgreSQL (see docker-compose.yml,
  database/init.sql). No code changes needed – SQLAlchemy async ORM + JSON
  columns abstract the dialect.
5.2 Tables (6)
Table
Purpose
Key columns
users
Accounts & roles
email (unique), full_name, hashed_password, role, is_active, is_verified
stress_assessments
Multi-modal assessment records
pss_q1–q10, total_pss, heart_rate, hrv_sdnn, sleep_*, work_*, screen_time, breaks, sentiment, anxiety, notes
predictions
ML predictions
predicted_class_id (0–3), stress_level, confidence_score, class_probabilities (JSON)
explainability_logs
XAI audit
shap_top_drivers (JSON), lime_rules (JSON)

rag_coping_logs
RAG audit
query_text, retrieved_interventions (JSON)
audit_logs
Security audit trail
action, ip_address, details (JSON)

5.3 Current data volume (live DB)
Table
Rows
users
10
stress_assessments
9
predictions
7
explainability_logs
5
rag_coping_logs
8
audit_logs
0

5.4 Key relationships
users 1–N stress_assessments 1–N predictions 1–1 explainability_logs
users 1–N rag_coping_logs (optional FK to predictions)
users 1–N audit_logs

---
6. ML Feature Engineering (25-dimension feature space)
#
Feature
Source
1–10
pss_q1 … pss_q10
Questionnaire (0–4 each)
11
total_pss
Server-side reverse scoring of items 4,5,7,8
12–16
heart_rate, hrv_sdnn, sleep_hours, sleep_efficiency, physical_activity_min
Physiological input
17–21
work_hours, screen_time_hours, breaks_per_day, sentiment_score, anxiety_score
Workload/cognitive input
22
hrv_to_hr_ratio
hrv_sdnn / heart_rate (sympathovagal balance)
23
sleep_deficiency_index
max(0, 8 − sleep_hours) × 0.7 + (1 − efficiency/100) × 3
24
work_stress_factor
(work_hours + screen_time) / (breaks × 2)
25
composite_strain_index
Weighted blend of PSS, anxiety, sentiment, sleep deficit

---
7. Deployment & Operations
Mode
Command

Endpoints
Local (recommended)
./start.sh
backend :8000, frontend :3000
Backend only
./start.sh backend
API docs at /docs
Frontend only
./start.sh frontend
web app
Retrain ML
./start.sh train
rebuilds model artifacts
Reset DB
rm stress_ai.db && ./start.sh seed
reseeds demo accounts
Docker
docker-compose up --build
Postgres + backend + frontend
Tests
pytest (37 tests)
API integration + ML pipeline

Environment configuration lives in .env (SECRET_KEY, DATABASE_URL, CORS origins, JWT expiry, NEXT_PUBLIC_API_URL).

---

8. Summary Diagram — Data Flow

Every prediction, explanation, and retrieval is persisted — making the system fully auditable end-to-end.